



INFORME DE GUÍA PRÁCTICA

I. PORTADA

Tema:	APE 3 ARBOLES
Unidad de Organización Curricular:	BÁSICA
Nivel y Paralelo:	TERCERO - B
Alumnos participantes:	Acurio López Paul Medardo
Asignatura:	Estructura de Datos
Docente:	Ing. Jose Caiza, Mg.

II. INFORME DE GUÍA PRÁCTICA

2.1 Objetivos

General:

Desarrollar soluciones algorítmicas en C++ y Java para la manipulación y análisis de árboles N-arios y binarios, aplicando operaciones de inserción, cálculo de métricas estructurales, recorridos DFS y transformaciones morfológicas de la estructura.

Específicos:

- Comprender y manipular la estructura básica de nodos con múltiples hijos y nodos binarios mediante el conteo de elementos.
- Implementar la lógica de inserción en un Árbol Binario de Búsqueda (BST) respetando sus propiedades de ordenamiento.
- Utilizar la recursividad para calcular métricas estructurales, específicamente la profundidad máxima o altura del árbol.
- Extraer datos mediante recorridos estándar aplicando la técnica In-Order.
- Modificar la estructura subyacente de los punteros para transformar un árbol en su versión invertida (espejo).

2.2 Modalidad

Presencial

2.3 Tiempo de duración

Presenciales: 8

No presenciales: 0

2.4 Instrucciones

- Lleve a cabo las actividades indicadas.
- Suba a plataforma un informe con el resultado obtenido.

2.5 Listado de equipos, materiales y recursos

Listado de equipos y materiales generales empleados en la guía práctica:

- Inteligencia artificial, TAC
- Apuntes de clase
- Internet
- Bibliografía virtual

TAC (Tecnologías para el Aprendizaje y Conocimiento) empleados en la guía práctica:

- ☒ Plataformas educativas
- ☐ Simuladores y laboratorios virtuales
- ☐ Aplicaciones educativas



- ☐ Recursos audiovisuales
- ☐ Gamificación
- ☐ Inteligencia Artificial
- Otros (Especifique): _____

2.6 Actividades por desarrollar

Descripción general

Las estructuras de datos jerárquicas permiten organizar información de forma que las operaciones de búsqueda y recorrido sean eficientes. Este proyecto se enfoca en representar y manipular datos mediante la implementación de cinco algoritmos fundamentales tanto en Java como en C++.

El sistema desarrollado abarca la lógica para iterar sobre árboles con múltiples hijos (N-arios), la inserción controlada de datos en un Árbol Binario de Búsqueda (BST), el cálculo de profundidad máxima, la extracción de datos mediante recorridos estándar (In-Order) y la modificación estructural de los enlaces para generar un árbol espejo.

Justificación académica

La APE es pertinente para la asignatura porque integra, en un entorno práctico, contenidos clave de Estructura de Datos: Tipos de Datos Abstractos (TDA), nodos enlazados, recursividad y manipulación de punteros en memoria. Al desarrollar los mismos algoritmos en C++ y Java, se refuerza la comprensión de cómo cada lenguaje maneja las referencias a los objetos en memoria. Además, se fomenta el pensamiento lógico al requerir que la solución se diseñe pensando en casos base y llamadas recursivas.

Explicación de la Estructura Utilizada

Para la representación de los árboles y la resolución de los ejercicios, la implementación técnica se fundamentó en los siguientes elementos:

- **Clases y Structs (Nodos):** Actúan como el bloque fundamental de la estructura. Para los árboles binarios, el nodo contiene un valor entero y dos referencias hacia los nodos descendientes (izquierda y derecha). Para el árbol N-ario, se utilizó una estructura dinámica (std::vector en C++ y List en Java) para almacenar un número indefinido de hijos.
- **Recursividad (DFS):** Los recorridos en profundidad (In-Order), el cálculo de la altura, el conteo total de nodos y la inversión del árbol se implementaron mediante funciones recursivas. Esto minimiza el código y utiliza la pila de ejecución del sistema para navegar por las ramas.
- **Validación de Inserción BST:** La función de inserción navega recursivamente comparando el valor a ingresar con el valor del nodo actual, bifurcando hacia la izquierda si es menor, o hacia la derecha si es mayor/igual, garantizando la integridad de un Árbol Binario de Búsqueda.

Definición de Conceptos Clave

Para comprender la funcionalidad de los algoritmos implementados, se definen los siguientes elementos:

- **Raíz:** Es el nodo de origen del cual se desprenden todos los demás. No posee un nodo superior.



- **Hoja:** Nodos en los extremos inferiores que no tienen hijos (ambas referencias apuntan a nulo).
- **Árbol N-ario:** Una estructura jerárquica donde cada nodo puede tener desde cero hasta N cantidad de hijos, a diferencia del binario que está limitado a dos.
- **Recorrido In-Order (DFS):** Estrategia que explora las ramas en el orden: Izquierda → Raíz → Derecha.
- **Altura del Árbol:** Es la longitud del camino más largo desde la raíz hasta una hoja. Se calcula sumando 1 al máximo entre las alturas de los subárboles izquierdo y derecho.

Análisis de Complejidad Temporal

La implementación de estas estructuras presenta los siguientes rendimientos:

- **Recorridos y Métricas (Conteo, Altura, In-Order, Inversión):** La complejidad es estrictamente de $O(n)$ ya que el algoritmo debe visitar obligatoriamente todos y cada uno de los nodos del árbol exactamente una vez.
- **Inserción en BST:** En un árbol balanceado, la complejidad de insertar un nuevo nodo es de $O(\log n)$. Sin embargo, si los datos ingresan de forma secuencial u ordenada, el árbol puede degenerar en una lista enlazada, afectando el rendimiento al peor caso $O(n)$.

2.7 Resultados obtenidos

Evidencia de Ejecución

El desarrollo colaborativo y los entregables se gestionan a través de control de versiones.

- **Código en C++**



UNIVERSIDAD TÉCNICA DE AMBATO
FACULTAD DE INGENIERÍA EN SISTEMAS ELECTRÓNICA E INDUSTRIAL
CARRERA DE SOFTWARE
CICLO ACADÉMICO: MARZO – JULIO 2025



```
Ejercicio1_Basico.cpp > ...
1  #include <iostream>
2  #include <vector>
3  using namespace std;
4
5  struct NodoN {
6      int valor;
7      vector<NodoN*> hijos;
8      NodoN(int v) : valor(v) {}
9  };
10
11 int contarNodos(NodoN* raiz) {
12     if (raiz == nullptr) {
13         return 0;
14     }
15
16     int contador = 1; // Contamos el nodo actual
17
18     // Iteramos sobre todos los hijos y sumamos recursivamente
19     for (NodoN* hijo : raiz->hijos) {
20         contador += contarNodos(hijo);
21     }
22
23     return contador;
24 }
25
26 int main() {
27     // Construimos el árbol
28     NodoN* n5 = new NodoN(5);
29     NodoN* n6 = new NodoN(6);
30     NodoN* n2 = new NodoN(2);
31     n2->hijos.push_back(n5);
32     n2->hijos.push_back(n6);
33     NodoN* n3 = new NodoN(3);
34     NodoN* n4 = new NodoN(4);
35     NodoN* raiz = new NodoN(1);
36     raiz->hijos.push_back(n2);
37     raiz->hijos.push_back(n3);
38     raiz->hijos.push_back(n4);
39
40     cout << "--- Prueba Ejercicio 1 ---" << endl;
41     cout << "Nodos esperados: 6" << endl;
42     cout << "Nodos calculados: " << contarNodos(raiz) << endl;
43
44     return 0;
45 }
```



UNIVERSIDAD TÉCNICA DE AMBATO
FACULTAD DE INGENIERÍA EN SISTEMAS ELECTRÓNICA E INDUSTRIAL
CARRERA DE SOFTWARE
CICLO ACADÉMICO: MARZO – JULIO 2025



```
1  #include <iostream>
2  using namespace std;
3
4  struct Nodo {
5      int valor;
6      Nodo* izquierdo;
7      Nodo* derecho;
8      Nodo(int v) : valor(v), izquierdo(nullptr), derecho(nullptr) {}
9  };
10
11  Nodo* insertar(Nodo* raiz, int valor) {
12      // Si llegamos a un nodo nulo, creamos el nuevo nodo aquí
13      if (raiz == nullptr) {
14          return new Nodo(valor);
15      }
16
17      // Si el valor es menor, vamos por la rama izquierda
18      if (valor < raiz->valor) {
19          raiz->izquierdo = insertar(raiz->izquierdo, valor);
20      }
21      // Si el valor es mayor, vamos por la rama derecha
22      else if (valor > raiz->valor) {
23          raiz->derecho = insertar(raiz->derecho, valor);
24      }
25
26      // Retornamos la raíz sin modificar
27      return raiz;
28  }
29
30  int main() {
31      Nodo* raiz = new Nodo(10);
32
33      insertar(raiz, 5);
34      insertar(raiz, 15);
35      insertar(raiz, 3);
36
37      cout << "--- Prueba Ejercicio 2 ---" << endl;
38
39      cout << "Raiz (Esperado 10): " << raiz->valor << endl;
40
41      if(raiz->izquierdo) cout << "Hijo Izquierdo (Esperado 5): " << raiz->izquierdo->valor << endl;
42      else cout << "Hijo Izquierdo: null" << endl;
43
44      if(raiz->derecho) cout << "Hijo Derecho (Esperado 15): " << raiz->derecho->valor << endl;
45      else cout << "Hijo Derecho: null" << endl;
46
47      if(raiz->izquierdo && raiz->izquierdo->izquierdo)
48          cout << "Hijo Izq del 5 (Esperado 3): " << raiz->izquierdo->izquierdo->valor << endl;
49      else cout << "Hijo Izq del 5: null" << endl;
50
51      return 0;
52  }
```



UNIVERSIDAD TÉCNICA DE AMBATO
FACULTAD DE INGENIERÍA EN SISTEMAS ELECTRÓNICA E INDUSTRIAL
CARRERA DE SOFTWARE
CICLO ACADÉMICO: MARZO – JULIO 2025



```
1 #include <iostream>
2 #include <algorithm> // util para std::max
3 using namespace std;
4
5 // Asumiendo la estructura Nodo del Ejercicio 2
6 struct Nodo {
7     int valor;
8     Nodo* izquierdo;
9     Nodo* derecho;
10    Nodo(int v) : valor(v), izquierdo(nullptr), derecho(nullptr) {}
11 };
12
13 int calcularAltura(Nodo* raiz) {
14     if (raiz == nullptr) {
15         return 0;
16     }
17
18     // Calculamos la altura de cada subárbol recursivamente usando std::max
19     return 1 + std::max(calcularAltura(raiz->izquierdo), calcularAltura(raiz->derecho));
20 }
21
22 int main() {
23     Nodo* raiz = new Nodo(1);
24     raiz->derecho = new Nodo(2);
25     raiz->derecho->izquierdo = new Nodo(3);
26
27     cout << "--- Prueba Ejercicio 3 ---" << endl;
28     cout << "Altura esperada: 3" << endl;
29     cout << "Altura calculada: " << calcularAltura(raiz) << endl;
30     cout << "Altura de arbol nulo (esperado 0): " << calcularAltura(nullptr) << endl;
31
32     return 0;
33 }
```

```
1 #include <iostream>
2 #include <vector>
3 using namespace std;
4
5 struct Nodo {
6     int valor;
7     Nodo* izquierdo;
8     Nodo* derecho;
9     Nodo(int v) : valor(v), izquierdo(nullptr), derecho(nullptr) {}
10 };
11
12 void inOrderAux(Nodo* nodo, vector<int>& resultado) {
13     if (nodo == nullptr) {
14         return;
15     }
16
17     // 1. Recorrer el subárbol izquierdo
18     inOrderAux(nodo->izquierdo, resultado);
19
20     // 2. Visitar el nodo actual (agregarlo al vector)
21     resultado.push_back(nodo->valor);
22
23     // 3. Recorrer el subárbol derecho
24     inOrderAux(nodo->derecho, resultado);
25 }
26
27 vector<int> recorridoInOrder(Nodo* raiz) {
28     vector<int> resultado;
29     inOrderAux(raiz, resultado);
30     return resultado;
31 }
32
33 int main() {
34     Nodo* raiz = new Nodo(4);
35     raiz->izquierdo = new Nodo(2);
36     raiz->derecho = new Nodo(6);
37     raiz->izquierdo->izquierdo = new Nodo(1);
```



```
38 raiz->izquierdo->derecho = new Nodo(3);
39 raiz->derecho->izquierdo = new Nodo(5);
40 raiz->derecho->derecho = new Nodo(7);
41
42 vector<int> res = recorridoInOrder(raiz);
43
44 cout << "--- Prueba Ejercicio 4 ---" << endl;
45 cout << "Resultado esperado: 1 2 3 4 5 6 7 " << endl;
46 cout << "Tu resultado: ";
47 for(int val : res) {
48     cout << val << " ";
49 }
50 cout << endl;
51
52 return 0;
53 }
```

```
1  #include <iostream>
2  using namespace std;
3
4  struct Nodo {
5      int valor;
6      Nodo* izquierdo;
7      Nodo* derecho;
8      Nodo(int v) : valor(v), izquierdo(nullptr), derecho(nullptr) {}
9  };
10
11 Nodo* invertir(Nodo* raiz) {
12     if (raiz == nullptr) {
13         return nullptr;
14     }
15
16     // Intercambiamos los hijos del nodo actual
17     Nodo* temporal = raiz->izquierdo;
18     raiz->izquierdo = raiz->derecho;
19     raiz->derecho = temporal;
20
21     // Llamamos recursivamente para invertir los subárboles
22     invertir(raiz->izquierdo);
23     invertir(raiz->derecho);
24
25     return raiz;
26 }
27
28 int main() {
29     Nodo* raiz = new Nodo(1);
30     raiz->izquierdo = new Nodo(2);
31     raiz->derecho = new Nodo(3);
32
33     cout << "--- Prueba Ejercicio 5 ---" << endl;
34     cout << "Antes de invertir:" << endl;
35     cout << "Hijo Izq: " << raiz->izquierdo->valor << " | Hijo Der: " << raiz->derecho->valor << endl;
```

```
36
37     invertir(raiz);
38
39     cout << "\nDespues de invertir (Esperado: Izq 3 | Der 2):" << endl;
40
41     if(raiz->izquierdo) cout << "Hijo Izq: " << raiz->izquierdo->valor;
42     else cout << "Hijo Izq: null";
43
44     cout << " | ";
45
46     if(raiz->derecho) cout << "Hijo Der: " << raiz->derecho->valor << endl;
47     else cout << "Hijo Der: null" << endl;
48
49     return 0;
50 }
```

- Código en Java



UNIVERSIDAD TÉCNICA DE AMBATO
FACULTAD DE INGENIERÍA EN SISTEMAS ELECTRÓNICA E INDUSTRIAL
CARRERA DE SOFTWARE
CICLO ACADÉMICO: MARZO – JULIO 2025



```
1  import java.util.ArrayList;
2  import java.util.Arrays;
3  import java.util.List;
4
5  static class NodoN { 16 usages  & Paul *
6      public int valor; 2 usages
7      public List<NodoN> hijos; 3 usages
8
9      @Contract(pure = true)
10     public NodoN(int valor) { 4 usages  & Paul
11         this.valor = valor;
12         this.hijos = new ArrayList<>();
13     }
14
15     @Contract(pure = true)
16     public NodoN(int valor, List<NodoN> hijos) { 2 usages  & Paul
17         this.valor = valor;
18         this.hijos = hijos;
19     }
20
21     @Contract(pure = true)
22     public static int contarNodos(NodoN raiz) { 2 usages  & Paul
23         if (raiz == null) {
24             return 0;
25         }
26
27         int contador = 1; // Contamos el nodo actual
28
29         // Iteramos sobre todos los hijos y sumamos recursivamente
30         for (NodoN hijo : raiz.hijos) {
31             contador += contarNodos(hijo);
32         }
33
34         return contador;
35     }
36
37     public static void main(String[] args) { & Paul
38         // Construimos el árbol:
39         //      1
40         //     / | \
41         //    2 3 4
42         //   / \
43         //  5  6
44         NodoN n5 = new NodoN(valor: 5);
45         NodoN n6 = new NodoN(valor: 6);
46         NodoN n2 = new NodoN(valor: 2, Arrays.asList(n5, n6));
47         NodoN n3 = new NodoN(valor: 3);
48         NodoN n4 = new NodoN(valor: 4);
49         NodoN raiz = new NodoN(valor: 1, Arrays.asList(n2, n3, n4));
50
51         System.out.println("--- Prueba Ejercicio 1 ---");
52         System.out.println("Nodos esperados: 6");
53         System.out.println("Nodos calculados: " + contarNodos(raiz));
54     }
```



UNIVERSIDAD TÉCNICA DE AMBATO
FACULTAD DE INGENIERÍA EN SISTEMAS ELECTRÓNICA E INDUSTRIAL
CARRERA DE SOFTWARE
CICLO ACADÉMICO: MARZO – JULIO 2025



```
1 class Nodo { 28 usages  ⚡ Paul
2     int valor; 12 usages
3     Nodo izquierdo, derecho; 24 usages
4     @Contract(pure = true)
5     Nodo(int valor) { this.valor = valor; } 15 usages  ⚡ Paul
6 }
7
8 public class Ejercicio2_Binario {  ⚡ Paul
9     @Contract("null, -> new")
10    public static Nodo ! insertar(Nodo raiz, int valor) { 5 usages  ⚡ Paul
11        // Si llegamos a una posición nula, creamos y retornamos el nuevo nodo
12        if (raiz == null) {
13            return new Nodo(valor);
14        }
15
16        // Si el valor es menor, recursión hacia la izquierda
17        if (valor < raiz.valor) {
18            raiz.izquierdo = insertar(raiz.izquierdo, valor);
19        }
20
21        // Si el valor es mayor o igual, recursión hacia la derecha
22        else if (valor >= raiz.valor) {
23            raiz.derecho = insertar(raiz.derecho, valor);
24        }
25
26        return raiz;
27    }
28
29    public static void main(String[] args) {  ⚡ Paul
30        Nodo raiz = new Nodo( valor: 10);
31
32        // Insertamos valores
33        insertar(raiz, valor: 5);
34
35        insertar(raiz, valor: 15);
36        insertar(raiz, valor: 3);
37
38        System.out.println("--- Prueba Ejercicio 2 ---");
39        System.out.println("Raiz (Esperado 10): " + raiz.valor);
40
41        String izq = (raiz.izquierdo != null) ? String.valueOf(raiz.izquierdo.valor) : "null";
42        System.out.println("Hijo Izquierdo de Raiz (Esperado 5): " + izq);
43
44        String der = (raiz.derecho != null) ? String.valueOf(raiz.derecho.valor) : "null";
45        System.out.println("Hijo Derecho de Raiz (Esperado 15): " + der);
46
47        String izqIzq = (raiz.izquierdo != null && raiz.izquierdo.izquierdo != null) ? String.valueOf(raiz.izquierdo.izquierdo.valor) :
48        System.out.println("Hijo Izquierdo del 5 (Esperado 3): " + izqIzq);
49    }
50 }
```

```
1 public class Ejercicio3_Binario2 {  ⚡ Paul
2     public static int calcularAltura(Nodo raiz) { 4 usages  ⚡ Paul
3         if (raiz == null) {
4             return 0;
5         }
6
7         // Calculamos la altura máxima entre el subárbol izquierdo y el derecho
8         return 1 + Math.max(calcularAltura(raiz.izquierdo), calcularAltura(raiz.derecho));
9     }
10
11    public static void main(String[] args) {  ⚡ Paul
12        //      1
13        //      \
14        //      2
15        //      /
16        //      3
17        Nodo raiz = new Nodo( valor: 1);
18        raiz.derecho = new Nodo( valor: 2);
19        raiz.derecho.izquierdo = new Nodo( valor: 3);
20
21        System.out.println("--- Prueba Ejercicio 3 ---");
22        System.out.println("Altura esperada: 3");
23        System.out.println("Altura calculada: " + calcularAltura(raiz));
24        System.out.println("Altura de árbol nulo (esperado 0): " + calcularAltura( raiz: null));
25    }
26 }
```



UNIVERSIDAD TÉCNICA DE AMBATO
FACULTAD DE INGENIERÍA EN SISTEMAS ELECTRÓNICA E INDUSTRIAL
CARRERA DE SOFTWARE
CICLO ACADÉMICO: MARZO – JULIO 2025



```
1 public class Ejercicio5_Transformacion { @Paul
2     @Contract("null -> null")
3     public static Nodo invertir(Nodo raiz) { 3 usages @Paul
4         if (raiz == null) {
5             return null;
6         }
7
8         // Intercambiamos los nodos hijos
9         Nodo temporal = raiz.izquierdo;
10        raiz.izquierdo = raiz.derecho;
11        raiz.derecho = temporal;
12
13        // Llamada recursiva para invertir los siguientes niveles
14        invertir(raiz.izquierdo);
15        invertir(raiz.derecho);
16
17        return raiz;
18    }
19
20    public static void main(String[] args) { @Paul
21        //      1          1
22        //    /  \      --> /  \
23        //   2   3         3   2
24        Nodo raiz = new Nodo( valor: 1);
25        raiz.izquierdo = new Nodo( valor: 2);
26        raiz.derecho = new Nodo( valor: 3);
27
28        System.out.println("--- Prueba Ejercicio 5 ---");
29        System.out.println("Antes de invertir:");
30        System.out.println("Hijo Izq: " + raiz.izquierdo.valor + " | Hijo Der: " + raiz.derecho.valor);
31
32        invertir(raiz);
33
34        System.out.println("\nDespués de invertir (Esperado: Izq 3 | Der 2):");
35        String izq = (raiz.izquierdo != null) ? String.valueOf(raiz.izquierdo.valor) : "null";
36        String der = (raiz.derecho != null) ? String.valueOf(raiz.derecho.valor) : "null";
37
38        System.out.println("Hijo Izq: " + izq + " | Hijo Der: " + der);
39    }
40 }
```



UNIVERSIDAD TÉCNICA DE AMBATO
FACULTAD DE INGENIERÍA EN SISTEMAS ELECTRÓNICA E INDUSTRIAL
CARRERA DE SOFTWARE
CICLO ACADÉMICO: MARZO – JULIO 2025



```
1 import java.util.ArrayList;
2 import java.util.List;
3
4 public class RecorridoInOrder { @ Paul
5     public static void inOrderAux(Nodo nodo, List<Integer> resultado) { 3 usages @ Paul
6         if (nodo == null) {
7             return;
8         }
9
10        // 1. Recorrer el subárbol izquierdo
11        inOrderAux(nodo.izquierdo, resultado);
12
13        // 2. Agregar el valor del nodo actual a la lista
14        resultado.add(nodo.valor);
15
16        // 3. Recorrer el subárbol derecho
17        inOrderAux(nodo.derecho, resultado);
18    }
19
20    @ public static List<Integer> recorridoInOrder(Nodo raiz) { 1 usage @ Paul
21        List<Integer> resultado = new ArrayList<>();
22        inOrderAux(raiz, resultado);
23        return resultado;
24    }
25
26    public static void main(String[] args) { @ Paul
27        //      4
28        //    /  \
29        //   2    6
30        //  / \  / \
31        // 1  3 5  7
32        Nodo raiz = new Nodo( valor: 4);
33
34        raiz.izquierdo = new Nodo( valor: 2);
35        raiz.derecho = new Nodo( valor: 6);
36        raiz.izquierdo.izquierdo = new Nodo( valor: 1);
37        raiz.izquierdo.derecho = new Nodo( valor: 3);
38        raiz.derecho.izquierdo = new Nodo( valor: 5);
39        raiz.derecho.derecho = new Nodo( valor: 7);
40
41        System.out.println("--- Prueba Ejercicio 4 ---");
42        System.out.println("Resultado esperado: [1, 2, 3, 4, 5, 6, 7]");
43        System.out.println("Tu resultado: " + recorridoInOrder(raiz));
44    }
```

- **Ejecución**

1. Ejercicio 1

```
--- Prueba Ejercicio 1 ---
Nodos esperados: 6
Nodos calculados: 6
* Las tareas reutilizarán el terminal, presione cualquier tecla para cerrarlo.
```

2. Ejercicio 2:

```
--- Prueba Ejercicio 2 ---
Raiz (Esperado 10): 10
Hijo Izquierdo (Esperado 5): 5
Hijo Derecho (Esperado 15): 15
Hijo Izq del 5 (Esperado 3): 3
* Las tareas reutilizarán el terminal, presione cualquier tecla para cerrarlo.
```

3. Ejercicio 3:



```
--- Prueba Ejercicio 3 ---  
Altura esperada: 3  
Altura calculada: 3  
Altura de arbol nulo (esperado 0): 0  
* Las tareas reutilizarán el terminal, presione cualquier tecla para cerrarlo.
```

4. Ejercicio 4:

```
--- Prueba Ejercicio 4 ---  
Resultado esperado: 1 2 3 4 5 6 7  
Tu resultado:      1 2 3 4 5 6 7  
* Las tareas reutilizarán el terminal, presione cualquier tecla para cerrarlo.
```

5. Ejercicio 5:

```
--- Prueba Ejercicio 5 ---  
Antes de invertir:  
Hijo Izq: 2 | Hijo Der: 3  
  
Despues de invertir (Esperado: Izq 3 | Der 2):  
Hijo Izq: 3 | Hijo Der: 2  
* Las tareas reutilizarán el terminal, presione cualquier tecla para cerrarlo.
```

2.8 Habilidades blandas empleadas en la práctica

- ☐ Liderazgo
- ☐ Trabajo en equipo
- ☒ Comunicación asertiva
- ☐ La empatía
- ☒ Pensamiento crítico
- ☐ Flexibilidad
- ☐ La resolución de conflictos
- ☐ Adaptabilidad
- ☒ Responsabilidad

2.9 Conclusiones

- La implementación de una función de búsqueda previa a la inserción en un Árbol Binario de Búsqueda (BST) no solo es una regla teórica, sino una medida de integridad de datos. Se comprobó que esta validación previene la redundancia de información, asegurando que cada nodo en la estructura jerárquica sea único, lo cual es crítico para aplicaciones de bases de datos o sistemas de inventarios donde los identificadores no deben repetirse.
- A través de la práctica, se determinó que el recorrido BFS, apoyado en la estructura de una cola (Queue), es la estrategia más coherente para desplegar menús de navegación en sistemas jerárquicos. Al procesar los nodos nivel por nivel, se facilita la visualización de categorías principales antes que los detalles profundos, simulando la lógica de navegación de un usuario en un Sistema Web real.

2.10 Recomendaciones

- Tener en cuenta las condiciones específicas de cada situación para decidir qué estructura de datos utilizar. Por ejemplo, se debe evaluar si el problema requiere búsquedas rápidas y validación de duplicados (donde un Árbol Binario de Búsqueda



es ideal) o si se necesita mapear jerarquías asimétricas, como sistemas de archivos (donde un árbol N-ario es indispensable).

- Considerar el impacto del orden de los datos de entrada en el rendimiento de un Árbol Binario de Búsqueda. Realizar pruebas de estrés insertando datos de forma secuencial para observar la degradación temporal al peor caso $O(n)$, y como trabajo futuro, investigar sobre árboles auto-balanceables (como los árboles AVL) para mitigar este problema en aplicaciones a gran escala.
- Al trabajar simultáneamente con implementaciones en C++ y Java, es fundamental prestar atención a las diferencias en la gestión de memoria. Adquirir la buena práctica de crear métodos para liberar explícitamente la memoria en C++, comprendiendo que Java maneja este proceso de forma automática mediante su recolector de basura.

2.11 Referencias bibliográficas

[1] A. Drozdek, Estructuras de datos y algoritmos en Java, 4.^a ed. México D.F., México: Cengage Learning, 2012.

[2] R. Lafore, Data Structures and Algorithms in Java, 2.^a ed. Indianapolis, IN, EE. UU.: Sams Publishing, 2017.

[3] R. Sedgewick y K. Wayne, Algorithms, 4.^a ed. Upper Saddle River, NJ, EE. UU.: Addison-Wesley Professional, 2011.

2.12 Anexos

Enlace de GitHub: https://github.com/Paul-Acurio/APE3_ARBOLES.git